

Intro to Data Structures with STL

Sean

October 10, 2026

Outline

- STL and iterator
- pair **and** tuple
- array
- vector **and** deque
- stack **and** queue
- priority_queue
- set **and** multiset
- map
- unordered_set **and** unordered_map



What is a data structure?

- A data structure is a systematic way to store data
- Different data structures help with different problems because they change the complexity of the operations you need
- Arrays are already a type of data structure
- In competitive programming, harder problems often require stronger data structures to solve them cleanly



STL and iterator

- STL (Standard Template Library)
- A built-in C++ library that contains many different data structures
- Works with many different types (int, long long, double, char)
- Different STL components can also be nested together
- It includes containers, iterators, and algorithms
- So let us start with iterators
- For the details, see [cpp reference](#)



iterator

- An iterator is a tool used to access elements inside STL containers
- To get the value pointed to by an iterator, put the dereference operator (*) in front
- The iterator categories we will use here are random-access, forward, and bidirectional iterators
 - Random-access iterator: like array indexing with `a[i]`; a random-access iterator can move to any position
 - Forward iterator: it can only move one position forward at a time (`++`)
 - Bidirectional iterator: it can move one position either forward or backward at a time (`--` / `++`)



iterator

- STL has two iterator types we often use: `::iterator` and `::reverse_iterator`
 - With `::iterator`, use `begin()` and `end()` for forward traversal
 - With `::reverse_iterator`, use `rbegin()` and `rend()` for reverse traversal
 - STL container ranges are left-closed and right-open, so `end()` and `rend()` cannot be dereferenced; for a nonempty vector, use `v.back()` to get the last element



Headers in competitive programming

- `bits/stdc++.h` includes almost everything you usually need in CP: containers, algorithms, strings, math, and I/O
- It is a GNU `libstdc++` convenience header, not standard C++; use explicit headers if your toolchain does not provide it

```
1 #include <bits/stdc++.h>  
2 using namespace std; // separate: lets us omit std::
```

Appendix: what does `bits/stdc++.h` include?

Appendix: GNU extensions and `bits/extc++.h`



iterator operations in code

```
1  #include <vector>
2  vector<int> v = {10, 20, 30};
3  auto it = v.begin();           // points to 10
4  ++it;                          // points to 20
5  *it = 25;                      // v = {10, 25, 30}
6  for (auto i = v.begin(); i != v.end(); ++i)
7      cout << *i << ' ';       // 10 25 30
8  for (auto i = v.rbegin(); i != v.rend(); ++i)
9      cout << *i << ' ';       // 30 25 10
10 if (!v.empty()) cout << v.back(); // 30
11 // end() and rend() are sentinels: never dereference.
12 // Iterator arithmetic (it + 1) needs random access.
```

C++17; assume using namespace std; and <iostream>.



STL

- STL can be roughly divided into four categories
- Sequence containers – array, vector, list, deque
- Container adaptors – stack, queue, priority_queue
- Associative containers – set, map, multiset, multimap
- Unordered associative containers – take the above and add unordered_ in front
- Next, we will go through these STL components one by one



pair

- It is in the header `#include <utility>`
- Declaration: `pair<T1, T2> p`
- Assignment: `make_pair(T1 a, T2 b)`
- After C++11, you can also use braces instead
- `pair<T1, T2> p = {a, b}`
- Use `p.first` and `p.second` to get the first and second values
- `pair` can be compared; by default it compares the first component first, and if they are equal, it compares the second
- You can also define your own `compare` function
- Small trick: if you want to sort by the second component, you can swap the first and second components when storing the data



pair operations in code

```
1  #include <utility>
2  pair<int, int> p = {2, 5};
3  p.first = 3;           // p = {3, 5}
4  cout << p.second;      // 5
5  auto q = make_pair(3, 7);
6  cout << (p < q);       // true: 3 == 3, then 5 < 7
7  auto [a, b] = p;       // C++17: copies the fields
8  a = 9;                 // p.first is still 3
9  auto& [x, y] = p;      // references to the fields
10 y = 8;                 // p = {3, 8}
11 swap(p, q);            // p = {3, 7}, q = {3, 8}
12 // Comparison uses first, then second; not second alone.
```



tuple

- What if your data needs three components?
- Nest pairs inside pairs?
~~pair<pair<int, int>, int>~~
- What if you need even more?
- Then you need tuple
- tuple and pair are actually very similar

tuple

- It is in the header `#include <tuple>`
- Declaration: `tuple<T1, T2, ..., Tn> t`
- Assignment: just replace `make_pair` with `make_tuple`
- You can use `get<i>(t)` to get the value of `t` at index `i`
- If you think that is too annoying, there are two small tricks
 - Use `tie(a, b, c, ...) = t` so that `a, b, c, ...` correspond to the values in `t`
 - After C++17, you can use `auto [a, b, c, ...] = t` to achieve the same effect
- Sorting works the same way as for `pair`



tuple operations in code

```
1  #include <tuple>
2  tuple<int, int, int> t = {2, 5, 8};
3  get<1>(t) = 6;           // t = {2, 6, 8}
4  auto u = make_tuple(2, 6, 9);
5  cout << (t < u);        // true: third field breaks tie
6  int a, b, c;
7  tie(a, b, c) = t;        // a = 2, b = 6, c = 8
8  tie(a, ignore, c) = u;   // a = 2, c = 9; b unchanged
9  auto [x, y, z] = t;      // C++17: copies
10 auto& [rx, ry, rz] = t;   // C++17: references
11 rz = 10;                 // changes get<2>(t) to 10
12 // get<i>: i is zero-based and known at compile time.
```



array

- It is in the header `#include <array>`
- Declaration: `array<T, size> a`
- Assignment: `a = {1, 42, 48763}`
- There are two ways to access elements
 - `a[1] = 42;`
 - `a.at(1) = 42;`
- `at()` performs bounds checking, and when you go out of range it will throw an exception
- `[]` is faster than `at()` for access



array

- A C-Style array, that is, something like `int a[5]`
- can decay into a pointer, so when you pass it to a function you may lose the size information
- But an STL array can use `.size()` to get its size
- Both `std::array` and C-style arrays with known bounds support range-based for loops
- `for (auto &it : a) std::cout << it;`



array operations in code

```
1  #include <array>
2  #include <algorithm>
3  array<int, 4> a{};           // {0, 0, 0, 0}
4  a.fill(7);                  // {7, 7, 7, 7}
5  a[1] = 2;
6  a.at(2) = 5;                // checks the index
7  cout << a.size();           // 4: size is fixed
8  cout << a.front() << ' ' << a.back(); // 7 7
9  sort(a.begin(), a.end()); // {2, 5, 7, 7}
10 for (int x : a) cout << x << ' ';
11 // a[4] is out of bounds; a.at(4) throws.
12 // array<int, 4> a; does not initialize its ints.
13 // No push_back(), pop_back(), or resize().
```



vector

- A dynamic array: an array whose length can grow and shrink freely
- It is one of the most frequently used STL containers
- It is in the header `#include <vector>`
- Declaration: `vector<T> v`
- `v.size()`: returns the length of `v`, complexity $\mathcal{O}(1)$
- `v.empty()` returns a `bool` telling whether `v` is empty, complexity $\mathcal{O}(1)$
- `v.push_back(T a)` adds `a` to the back of `v`, amortized complexity $\mathcal{O}(1)$
- `v.pop_back()` removes the element at the back of `v`, complexity $\mathcal{O}(1)$



vector

- `v.clear()` clears `v`, complexity $\mathcal{O}(\text{size})$
- `v.resize(size_type n)` changes the length of `v` to `n`
- `v.assign(size_type n, T a)` turns `v` into an array of length `n` where every element is `a`
- When `vector` runs out of capacity, its implementation usually doubles the size of the storage, so sometimes `vector` can be slower than a plain array
- `string` is actually a lot like `vector<char>`, so `string` supports many operations that `vector` has
- But it also has some extra optimizations, for example:
 - `s += t` concatenates two strings
 - `getline(cin, s)`



vector access and updates

```
1  #include <vector>
2  vector<int> v = {2, 4, 6};
3  v.push_back(8);           // {2, 4, 6, 8}
4  v.emplace_back(10);       // {2, 4, 6, 8, 10}
5  cout << v.size();         // 5
6  v[1] = 5;                 // {2, 5, 6, 8, 10}
7  cout << v.at(2);          // 6; checks the index
8  if (!v.empty()) {
9      cout << v.front() << ' ' << v.back(); // 2 10
10     v.pop_back();          // removes 10; returns nothing
11 }
12 for (int& x : v) ++x;      // {3, 6, 7, 9}
13 // [] needs a valid index; at() throws if out of range.
14 // front(), back(), pop_back() require nonempty v.
```



vector size and insertion

```
1  #include <vector>
2  vector<int> v = {2, 4, 6};
3  v.reserve(100);           // capacity >= 100; size is 3
4  v.resize(5, 9);           // {2, 4, 6, 9, 9}
5  v.resize(3);              // {2, 4, 6}
6  v.insert(v.begin() + 1, 7); // {2, 7, 4, 6}
7  v.erase(v.begin() + 2);   // {2, 7, 6}
8  v.erase(v.begin(), v.begin() + 2); // {6}; [first,last)
9  v.assign(3, 5);           // {5, 5, 5}
10 v.clear();                // size 0; keeps capacity
11 cout << v.empty();       // true
12 // reserve() adds no elements: v[0] is invalid here.
13 // Reallocation invalidates all iterators/references.
14 // insert/erase: invalidates at and after position.
15 // Middle insertion/erasure shifts elements: O(size).
```



string operations in code

```
1  #include <string>
2  string s = "abc";
3  s += "de";           // "abcde"
4  s.push_back('f');    // "abcdef"
5  s.pop_back();        // "abcde"; needs nonempty s
6  s[0] = 'A';          // "Abcde"
7  cout << s.size();    // 5
8  cout << s.substr(1, 3); // "bcd": start, length
9  auto pos = s.find("cd"); // 2
10 if (pos != string::npos) cout << pos;
11 s.erase(1, 2);       // "Ade": start, length
12 // find() returns string::npos when there is no match.
13 // After cin >> ..., consume the leftover newline before
14 // getline(cin, s); it reads spaces, unlike cin >> s.
```



deque

- It is in the header `#include <deque>`
- Think of it as a `vector` that can also push and pop at the front
- `dq.push_front()` and `dq.pop_front()` are for adding and removing elements at the front
- It supports efficient operations at both ends, but usually has worse constants than `vector`
- Use `deque` when you need front operations; otherwise, `vector` is usually the better default



deque operations in code

```
1  #include <deque>
2  deque<int> dq = {2, 4};
3  dq.push_front(1);           // {1, 2, 4}
4  dq.push_back(8);            // {1, 2, 4, 8}
5  cout << dq.front() << ' ' << dq.back(); // 1 8
6  dq[1] = 3;                  // {1, 3, 4, 8}
7  dq.pop_front();             // {3, 4, 8}
8  dq.pop_back();              // {3, 4}
9  cout << dq.size();          // 2
10 for (int x : dq) cout << x << ' '; // 3 4
11 dq.clear();                  // dq.empty() is true
12 // Check empty() before front/back/pop_front/pop_back.
13 // Pushing at either end invalidates existing iterators.
14 // [] is unchecked; at() checks the index.
```



stack and queue

- In general, containers like `std::stack`, `std::queue`, and `std::priority_queue` do not expose iterators
- These belong to container adaptor; they are meant to restrict how you access or modify elements
- So they provide a more limited interface for specific use cases
- instead of the more general iterator-based operations that other containers provide



stack

- Think of this container as a jar: each time, you can only take an item from the top or put an item onto the top
- In other words, 「 Last In, First Out 」 (LIFO)
- It is in the header `#include <stack>`
- Declaration: `stack<T> st`
- `st.size()` and `st.empty()` are the same as `vector`
- `st.top()` returns the most recently pushed element
- `st.push(T a)` pushes the element `a` into `st`
- `st.pop()` removes the most recently pushed element
- All of the above operations have complexity $\mathcal{O}(1)$



stack operations in code

```
1  #include <stack>
2  stack<int> st;
3  st.push(2);
4  st.push(5);           // 5 is on top
5  st.emplace(8);        // 8 is now on top
6  cout << st.size();    // 3
7  while (!st.empty()) {
8      int x = st.top();  // read before removing
9      st.pop();          // removes top; returns nothing
10     cout << x << ' '; // 8 5 2
11 }
12 // top() and pop() require a nonempty stack.
13 // No indexing, iterators, or clear(); pop to empty it.
```



queue

- Think of this container as a line: each time, people enter at the back of the line, or leave from the front
- In other words, 「 First In, First Out 」 (FIFO)
- It is in the header `#include <queue>`
- Declaration: `queue<T> q`
- `q.size()` and `q.empty()` are the same as `vector`
- `q.front()` returns the first element that was pushed
- `q.back()` returns the most recently pushed element
- `q.push(T a)` pushes the element `a` into `q`
- `q.pop()` removes the first element that was pushed
- All of the above operations have complexity $\mathcal{O}(1)$



queue operations in code

```
1  #include <queue>
2  queue<int> q;
3  q.push(2);
4  q.push(5);
5  q.emplace(8);           // 8 joins at the back
6  cout << q.size();       // 3
7  cout << q.back();       // 8
8  while (!q.empty()) {
9      int x = q.front();   // read before removing
10     q.pop();             // removes front; returns void
11     cout << x << ' ';   // 2 5 8
12 }
13 // front(), back(), pop() require a nonempty queue.
14 // No indexing or iterators; traversal above empties q.
```



queue

- You can also implement the functionality of a queue using `vector`
- A head index marks the front; popped elements stay in storage, so memory tracks total pushes

```
1  vector<int> q = {2, 5};
2  size_t head = 0;
3  q.push_back(8); // push()
4  cout << q.back() << '\n'; // back(): 8
5  while (head < q.size()) { // not empty
6      int x = q[head]; // front()
7      cout << x << '\n'; // 2, 5, 8
8      ++head; // pop()
9  }
```



priority_queue

- `priority_queue` is built from a binary heap, and is used to keep track of the current most extreme value
- Binary heap visualization
- It is in the header `#include <queue>`
- Declaration: `priority_queue<T, Container, cmp>`.
T is the type, Container is the underlying container, usually vector, and cmp is the comparison rule
 - `priority_queue<int, vector<int>, less<int>>`
pq: maximum value comes out first
 - `priority_queue<int, vector<int>, greater<int>>` pq: minimum value comes out first
- You can also build it from an iterator range, for example `priority_queue<int> pq(first, last)`, which inserts all elements in *[first, last)*



priority_queue

- `pq.size()`, `pq.empty()`, and `pq.top()` are used the same way as in other STL containers
- A simple rule: `less` gives a max-heap, and `greater` gives a min-heap
- `pq.push(x)` pushes `x` into `pq`, $\mathcal{O}(\log size)$
- `pq.pop()` removes the top element, $\mathcal{O}(\log size)$
- Building from a range is $\mathcal{O}(size)$, while pushing elements one by one is $\mathcal{O}(size \log size)$



priority_queue example

CSES 1161 Stick Divisions

Split a stick of length n into m pieces of lengths $d_1, d_2, \dots, d_m$. Each split costs the length of the stick being split. Find the minimum total cost. [link](#)

- Think in reverse: instead of splitting, think of merging
- Always merge the two smallest sticks
- Use a min-heap
- Read `top()` then `pop()`, twice. Add the sum to the cost and push it back. Repeat until one stick remains; use `long long` for the cost



priority_queue operations in code

```
1  #include <queue>
2  #include <vector>
3  #include <functional>
4  priority_queue<int> mx;  // max-heap by default
5  priority_queue<int, vector<int>, greater<int>> mn;
6  for (int x : {4, 1, 7}) {
7      mx.push(x);
8      mn.push(x);
9  }
10 cout << mx.top();      // 7; top() does not remove it
11 while (!mn.empty()) {
12     cout << mn.top() << ' '; // 1 4 7
13     mn.pop();           // removes top; returns nothing
14 }
15 // Check empty() before top/pop; no find/erase-by-value.
```



set

- `set` is built from a balanced binary search tree (Balanced Binary Search Tree), so in simple terms it is a tree-shaped data structure
- Binary Search Tree (BST) visualization
- It is similar to the mathematical notion of a set: it does not contain duplicate elements
- It supports insertion, deletion, and search in $\mathcal{O}(\log \text{size})$
- It is in the header `#include <set>`
- Declaration: `set<T> s`



set

- `s.size()` and `s.empty()` are the same as for other containers
- `s.insert(T val)` inserts `val` into `s`, $\mathcal{O}(\log \textit{size})$
- `s.find(T val)` finds the iterator pointing to `val`; if it does not exist, it returns `s.end()`, $\mathcal{O}(\log \textit{size})$
- `s.erase(T val)` erases `val` from `s`, $\mathcal{O}(\log \textit{size})$
- `s.erase(iterator first, iterator last)` erases the elements in $[first, last)$, $\mathcal{O}(\log \textit{size} + \textit{count})$
- `s.count(T val)` gets the number of occurrences of `val` in `s`, $\mathcal{O}(\log \textit{size} + \textit{count})$



set

- `s.lower_bound(T val)` finds the first iterator in `s` that points to an element $\geq val$, $\mathcal{O}(\log size)$
- `s.upper_bound(T val)` finds the first iterator in `s` that points to an element $> val$, $\mathcal{O}(\log size)$
- `set::iterator` is a bidirectional iterator, and the elements are kept in increasing order; moving the iterator forward or backward has amortized complexity $\mathcal{O}(1)$



set operations in code

```
1  #include <set>
2  set<int> s = {2, 5, 8};
3  s.insert(5);           // duplicate ignored; size 3
4  s.insert(6);           // {2, 5, 6, 8}
5  cout << s.count(5);    // 1; set count is 0 or 1
6  auto it = s.find(5);
7  if (it != s.end()) s.erase(it); // {2, 6, 8}
8  auto lo = s.lower_bound(6); // first >= 6
9  auto hi = s.upper_bound(6); // first > 6
10 if (lo != s.end()) cout << *lo; // 6
11 if (hi != s.end()) cout << *hi; // 8
12 for (int x : s) cout << x << ' '; // 2 6 8
13 s.erase(2);           // {6, 8}; absent value is OK
14 // Never dereference/erase end(); no s[i] or it + 1.
15 // Keys cannot be modified through a set iterator.
```



multiset

- `multiset` is a set that allows duplicate elements
- It is used exactly the same way as `set`
- If the argument passed to `s.erase(val)` is a value instead of an iterator, then it will erase every `val` in `s`
- So if you want to erase only one element, you need to pass an iterator; in implementations this is often used together with `lower_bound` and `upper_bound`



multiset example

CSES 1076 Sliding Median

Find the median of every subarray of length k . [link](#)

- Split the values into a lower half and an upper half
- For prefix medians, two heaps suffice: a max-heap for the lower half and a min-heap for the upper half
- For a sliding window, use two `multisets` so you can remove one occurrence of the outgoing value
- Rebalance so the lower half has $\lceil k/2 \rceil$ elements, all no larger than the upper half; its largest value is the median



multiset operations in code

```
1  #include <set>
2  multiset<int> ms = {2, 5, 5, 8};
3  ms.insert(5);           // {2, 5, 5, 5, 8}
4  cout << ms.count(5);    // 3
5  auto it = ms.find(5);
6  if (it != ms.end()) ms.erase(it); // remove ONE 5
7  cout << ms.count(5);    // 2
8  auto lo = ms.lower_bound(5); // first 5
9  auto hi = ms.upper_bound(5); // first value > 5
10 ms.erase(lo, hi);      // remove ALL 5s; {2, 8}
11 ms.insert(8);           // {2, 8, 8}
12 ms.erase(8);           // remove ALL 8s; {2}
13 if (!ms.empty()) cout << *ms.begin(); // minimum: 2
14 // Check find() != end() before erasing its iterator.
15 // erase(value) removes every copy, not just one.
```



map

- It consists of `key` and `value`; each `key` corresponds to exactly one `value`
- Like `set`, it is a tree structure and supports insertion, deletion, and search in $\mathcal{O}(\log \textit{size})$
- It is in the header `#include <map>`
- `map<T1, T2> mp` has keys of type `T1` and values of type `T2`; entries are ordered and unique by key
- `map` is actually a structure made of `pair`, so the element pointed to by an iterator is a `pair`; to get the key, use `first`, and to get the value, use `second`
- In competitive programming, `map` is useful when the keys are not small consecutive integers, so direct indexing with an array or `vector` is inconvenient



map

- Its general usage is the same as `set`
- `mp.insert({key, value})` inserts an entry; an existing key keeps its current value
- `mp[key]` accesses the value corresponding to `key`, $\mathcal{O}(\log \text{size})$
- `mp[key] = value` inserts or updates the value for `key`, $\mathcal{O}(\log \text{size})$
 - If `key` is missing, `mp[key]` inserts a value-initialized value: `0` for `int`, an empty string for `string`
- Typical uses: frequency counting on large or sparse values, string keys, and cases where direct indexing is inconvenient



map operations in code

```
1  #include <map>
2  #include <string>
3  map<string, int> mp;
4  ++mp["apple"];           // missing -> 0, then +1
5  mp["pear"] = 3;          // insert or update
6  mp.insert({"pear", 9});  // existing value stays 3
7  auto it = mp.find("banana"); // lookup without insertion
8  if (it != mp.end()) cout << it->second;
9  cout << mp.count("banana"); // 0; still absent
10 for (const auto& [key, value] : mp)
11     cout << key << ':' << value << ' '; // keys sorted
12 mp.erase("pear");        // erase by key
13 cout << mp["banana"];    // 0; now INSERTS "banana"
14 // at(key) reads without insertion; throws if absent.
15 // lower_bound/upper_bound work on keys, as in set.
```



map

- `multimap` is a less common version of `map` that allows multiple values for the same key
- Useful when duplicate keys must stay sorted, such as events by time, intervals by endpoint, or multiple records under one key

multimap operations in code

```
1  #include <map>
2  #include <string>
3  multimap<int, string> events;
4  events.insert({3, "A"});
5  events.insert({3, "B"}); // duplicate key allowed
6  events.emplace(5, "C");
7  cout << events.count(3); // 2
8  auto [first, last] = events.equal_range(3);
9  for (auto it = first; it != last; ++it)
10     cout << it->second << ' '; // A B
11  if (first != last) events.erase(first); // ONE record
12  events.erase(3); // ALL remaining records at 3
13  // No operator[] or at(): a key may have many values.
14  // find: one match; equal_range: all matching records.
```



unordered_set, unordered_map

- They use `hash`, so lookup, insertion, and deletion are average $\mathcal{O}(1)$
- The elements are not ordered, so you cannot use `lower_bound`, `upper_bound`, or `ordered` traversal
- Their iterators are forward iterators
- `hash` has collision risk, so worst-case performance can become much worse
- In competitive programming, use `unordered_*` when you want fast average lookup; use `set` or `map` when you need order or more predictable behavior
- In contests like `codeforces` where post-contest hacks exist, a custom `hash` function is often important



custom hash

```
1 struct custom_hash {
2     static uint64_t splitmix64(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6         return x ^ (x >> 31);
7     }
8
9     size_t operator()(uint64_t x) const {
10         static const uint64_t FIXED_RANDOM =
11             chrono::steady_clock::now()
12                 .time_since_epoch().count();
13         return splitmix64(x + FIXED_RANDOM);
14     }
15 };
```



unordered_set operations in code

```
1  #include <unordered_set>
2  unordered_set<int> seen;
3  seen.reserve(100);           // plan for about 100 elements
4  seen.insert(5);
5  seen.insert(5);              // duplicate ignored
6  seen.insert(2);
7  cout << seen.size();        // 2
8  cout << seen.count(5);      // 1
9  auto it = seen.find(2);
10 if (it != seen.end()) seen.erase(it); // remove 2
11 seen.erase(42);              // missing value: no effect
12 for (int x : seen) cout << x << ' '; // unordered
13 seen.clear();                 // seen.empty() is true
14 // No lower_bound/upper_bound; lookup is average O(1).
15 // Rehash invalidates iterators: find again afterward.
```



unordered_map operations in code

```
1  #include <unordered_map>
2  unordered_map<int, int> freq;
3  freq.reserve(100);           // reserve before inserting
4  for (int x : {7, 2, 7}) ++freq[x];
5  cout << freq[7];             // 2
6  auto it = freq.find(9);      // no insertion
7  if (it != freq.end()) cout << it->second;
8  cout << freq.count(9);       // 0
9  freq.insert({7, 99});        // existing value stays 2
10 for (const auto& [key, value] : freq)
11     cout << key << ':' << value << ' '; // unordered
12 freq.erase(2);               // erase by key
13 cout << freq[9];             // 0; INSERTS the missing key
14 // [] can insert; rehash invalidates iterators.
15 // Custom hash: unordered_map<int,int,custom_hash>
```



Choosing a container

- vector: dynamic array
- stack: LIFO
- queue: FIFO
- deque: operations at both ends
- priority_queue: current max/min
- set, multiset: ordered values
- map: ordered key-value
- unordered_*: hash-based, average $\mathcal{O}(1)$



The End

Thanks for listening ~

© 2026 Sean. Please do not repost or redistribute without permission.



Appendix: `bits/stdc++.h`

- A convenient collection of standard headers; examples include:
- **Containers:** `vector`, `array`, `deque`, `stack`, `queue`, `set`, `map`, `unordered_*`
- **Algorithms and utilities:** `algorithm`, `numeric`, `utility`, `tuple`, `bitset`
- **Strings, math, and I/O:** `string`, `cmath`, `iostream`, `iomanip`
- The exact list depends on the library version and selected C++ standard

Full include list: [GCC 15.1 source](#)

[Back to headers](#)

GNU extensions: `bits/extc++.h`



Appendix: `bits/extc++.h`

- In C++11 and later, it includes `bits/stdc++.h` plus GNU extensions
- Examples: `__gnu_pbds` (PBDS) and `__gnu_cxx::rope`
- Both headers are nonstandard C++

```
1  #include <bits/extc++.h> // standard + GNU extensions
2  // Or include just the PBDS headers you need:
3  // #include <ext/pb_ds/assoc_container.hpp>
4  // #include <ext/pb_ds/tree_policy.hpp>
```

Full include list: [GNU extension header source](#)

[Back to headers](#)

[bits/stdc++.h appendix](#)

